

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Artificial Intelligence Application Development

Module 1

Practical –II

Roll no. : T010	Name : Sagar kandari
Class : TYIT	Batch : 1
Date of assignment : 05/08/2026	Date of assignment : 05/08/2026

2. Using a suitable regression dataset, implement Linear Regression using scikit-learn. Code :

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

data = pd.read_csv("studentmarks.csv")
```

```
X = data[['hours']]
Y = data['marks']
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Display accuracy
print(
    "Training Accuracy:",
    round(model.score(X_train, y_train) * 100, 2)
)

print(
```

```
"Testing Accuracy:",  
round(model.score(X_test, y_test) * 100, 2)  
)  
  
# Predict marks for 11 study hours  
new_data = pd.DataFrame([[11]],  
columns=["hours"]) pred = model.predict(new_data)  
  
print(  
    "Predicted Marks for 11 hours:",  
    round(pred[0], 2)  
)
```

Output :

3. Using the MNIST dataset (digit vs not digit), implement a binary classifier.

Code : from sklearn.datasets import load_digits from
sklearn.model_selection import train_test_split from
sklearn.linear_model import LogisticRegression

```
# Load MNIST-like Digits dataset  
digits = load_digits()
```

```
# Features and target X  
= digits.data  
y = digits.target
```

```
# Binary classification  
# True if digit is 5, otherwise False y  
= (y == 5)
```

```
# Split dataset  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.3,  
    random_state=1  
)
```

```
# Train model
```

Sagar kandari
T010

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Display accuracy print("Training
Accuracy:",
round(model.score(X_train,
y_train) * 100, 2), "%")

print("Testing Accuracy:",
round(model.score(X_test, y_test) * 100, 2), "%")

# Predict first test sample
prediction = model.predict([X_test[0]])

if prediction[0]:
    print("Digit is 5") else:
    print("Digit is not 5")
```

Output :

```
Training Accuracy: 100.0 %
Testing Accuracy: 98.89 %
Digit is not 5
```

Task :

Q.1

Area (sq.ft.)	Price (Lakhs)
600	32
800	40
1000	49
1200	58
1400	68
1600	79

Find the regression equation.

Predict the price of a 1500 sq.ft. house.

Predict the price of a 2500 sq.ft. house.

Code :

```
import pandas as pd
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
```

Given Data

Sagar kandari
T010

```
data = {
    'Area': [600, 800, 1000, 1200, 1400, 1600],
    'Price': [32, 40, 49, 58, 68, 79]
}

df = pd.DataFrame(data)

# Independent and Dependent Variables
X = df[['Area']]
y = df['Price']

# Train Model
model = LinearRegression()
model.fit(X, y)

# Regression Equation
print("Slope (m):", model.coef_[0])
print("Intercept (c):", model.intercept_)

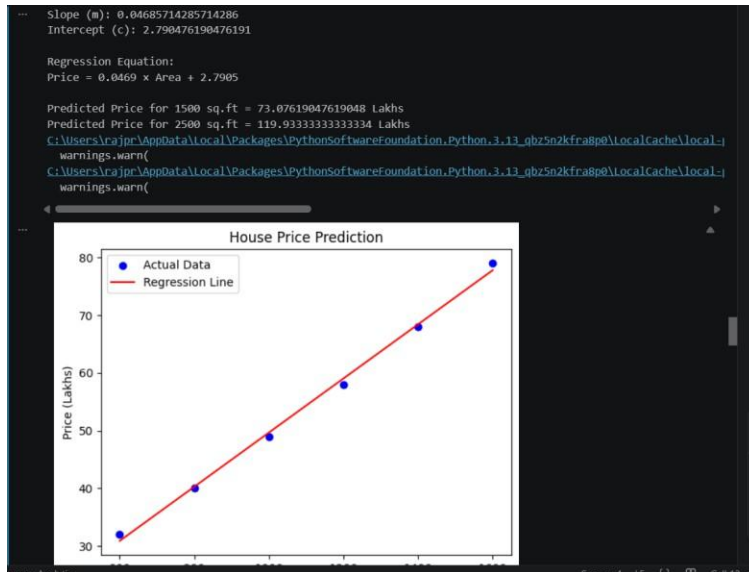
# Equation
print(f"\nRegression Equation:")
print(f"Price = {model.coef_[0]:.4f} × Area + {model.intercept_:.4f}")

# Predictions
price1500 = model.predict([[1500]])
price2500 = model.predict([[2500]])

print("\nPredicted Price for 1500 sq.ft =", price1500[0], "Lakhs")
print("Predicted Price for 2500 sq.ft =", price2500[0], "Lakhs")

# Plot
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, model.predict(X), color='red', label='Regression Line')
plt.xlabel("Area (sq.ft.)")
plt.ylabel("Price (Lakhs)")
plt.title("House Price Prediction")
plt.legend()
plt.show()
```

Output :



Q.2 A company has collected the following employee data:

- Employee ID
- Employee Age
- Years of Experience
- Salary

Code :

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Employee Data
data = {
    'Employee_ID': [101,102,103,104,105,106,107,108,109,110],
    'Age': [22,25,28,30,35,40,45,50,32,38],
    'Experience': [1,2,3,5,7,10,12,15,6,9],
    'Salary':
[25000,30000,35000,45000,60000,75000,85000,100000,55000,70000]
}

df= pd.DataFrame(data)

print(df)

# Independent Variable
X = df[['Experience']]

# Dependent Variable
y = df['Salary']
```

Sagar kandari
T010

Split Data

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

Train Model

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

Predict Salary for 7 Years Experience

```
salary = model.predict([[7]])
```

```
print("\nPredicted Salary for 7 Years Experience =", salary[0])
```

Results

```
print("\nCoefficient =", model.coef_[0])
```

```
print("Intercept =", model.intercept_)
```

```
print("R2 Score =", model.score(X_test, y_test))
```

Plot

```
plt.scatter(X, y, color='blue')
```

```
plt.plot(X, model.predict(X), color='red')
```

```
plt.xlabel("Years of Experience")
```

```
plt.ylabel("Salary")
```

```
plt.title("Salary Prediction")
```

```
plt.show()
```

Output :

```
...   Employee_ID  Age  Experience  Salary  
0      101      22         1    25000  
1      102      25         2    30000  
2      103      28         3    35000  
3      104      30         5    45000  
4      105      35         7    60000  
5      106      40        10    75000  
6      107      45        12    85000  
7      108      50        15   100000  
8      109      32         6    55000  
9      110      38         9    70000  
  
Predicted Salary for 7 Years Experience = 57776.87296416938  
  
Coefficient = 5464.169381107491  
Intercept = 19527.687296416945  
R2 Score = 0.9763255202442528
```



1. California Housing Dataset

Problem Statement

A real estate company wants to predict the median house value based on housing features.

Code :

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

Load Dataset

```
df = pd.read_csv("California_houses.csv")
```

Display First Five Records

```
print("First Five Records:")
print(df.head())
```

Check Missing Values

```
print("\nMissing Values:")
print(df.isnull().sum())
```

Independent Variable (Median Income)

```
X = df[['Median_House_Value']]
```

Dependent Variable (Median House Value)

```
y = df['Median_Income']
```

Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Train Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

Predict House Prices

```
predictions = model.predict(X_test)
```

```
print("\nFirst 10 Predicted House Prices:")
```

```
print(predictions[:10])
```

Sagar kandari
T010

```
# Display Results
print("\nCoefficient =", model.coef_[0])
print("Intercept =", model.intercept_)
print("R2 Score =", model.score(X_test, y_test))

# Plot Regression Line
plt.scatter(X_test, y_test, color='blue', label='Actual')

plt.plot(
    X_test,
    model.predict(X_test),
    color='red',
    linewidth=2,
    label='Regression Line'
)
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("California Housing Price Prediction")
plt.legend()
plt.show()

# Interpretation
print("\nInterpretation:")
print("- Positive coefficient means higher median income generally leads to higher house value.")
print("- R2 Score indicates how well the model explains the variation in house prices.")
print("- The regression line shows the relationship between median income and house value.")
```

Output:

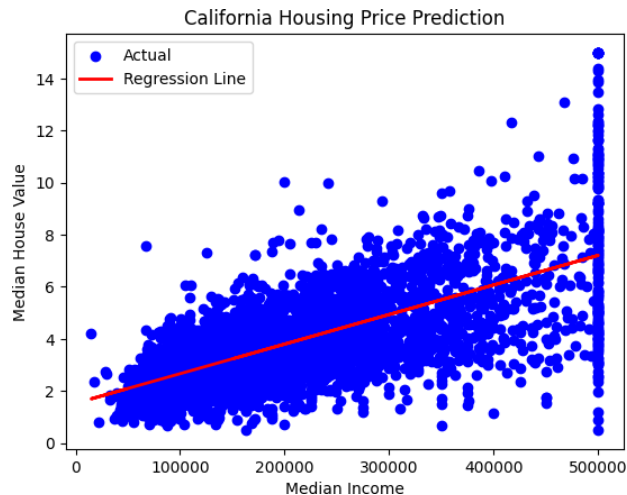
```
... First Five Records:
  Median_House_Value  Median_Income  Median_Age  Tot_Rooms  Tot_Bedrooms  \
0      452600.0         8.3252         41         880         129
1      358500.0         8.3014         21        7099        1106
2      352100.0         7.2574         52        1467         190
3      341300.0         5.6431         52        1274         235
4      342200.0         3.8462         52        1627         280

  Population  Households  Latitude  Longitude  Distance_to_coast  \
0         322         126      37.88     -122.23      9263.040773
1        2401        1138      37.86     -122.22     10225.733072
2         496         177      37.85     -122.24      8259.085109
3         558         219      37.85     -122.25      7768.086571
4         565         259      37.85     -122.25      7768.086571

  Distance_to_LA  Distance_to_SanDiego  Distance_to_SanJose  \
0  556529.158342      735501.806984      67432.517001
1  554279.850069      733236.884360      65049.908574
2  554610.717069      733525.682937      64867.289833
3  555194.266086      734095.290744      65287.138412
4  555194.266086      734095.290744      65287.138412

  Distance_to_SanFrancisco
0      21250.213767
1     20880.600400
...

Coefficient = 1.137488547980605e-05
Intercept = 1.5239383442177306
R2 Score = 0.458520037302225
```



Q.1 Modify the program to detect Digit 7 vs Not Digit 7 instead of Digit 5 vs Not Digit 5.

Code :

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
```

```
# Load MNIST Dataset
```

```
mnist = fetch_openml('mnist_784', version=1, as_frame=False)
```

```
X = mnist.data
```

```
y = mnist.target.astype(int)
```

```
# Digit 7 vs Not 7
```

```
y_7 = (y == 7)
```

```
# Split Dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y_7, test_size=0.2, random_state=42
)
```

```
# Train Model
```

```
model = SGDClassifier(random_state=42)
model.fit(X_train, y_train)
```

```
# Prediction
```

```
y_pred = model.predict(X_test)
```

Sagar kandari
T010

Results

```
print("Accuracy:", accuracy_score(y_test, y_pred))  
print("\nConfusion Matrix")  
print(confusion_matrix(y_test, y_pred))
```

```
print("\nClassification Report")  
print(classification_report(y_test, y_pred))
```

Output :

```
... Accuracy: 0.9730714285714286  
  
Confusion Matrix  
[[12438   59]  
 [   318 1185]]  
  
Classification Report  
precision    recall  f1-score   support  
  
   False     0.98     1.00     0.99     12497  
    True     0.95     0.79     0.86     1503  
  
 accuracy          0.97     14000  
macro avg          0.96     0.89     0.92     14000  
weighted avg          0.97     0.97     0.97     14000
```

Q.2 Modify the program to detect Digit 0 vs Not Digit 0.

Code :

```
from sklearn.datasets import fetch_openml  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import SGDClassifier  
from sklearn.metrics import accuracy_score, confusion_matrix,  
classification_report
```

Load Dataset

```
mnist = fetch_openml('mnist_784', version=1, as_frame=False)
```

```
X = mnist.data  
y = mnist.target.astype(int)
```

Digit 0 vs Not 0

```
y_0 = (y == 0)
```

Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y_0, test_size=0.2, random_state=42  
)
```

Train Model

```
model = SGDClassifier(random_state=42)  
model.fit(X_train, y_train)
```

Prediction

Sagar kandari
T010

```
y_pred = model.predict(X_test)
```

```
# Results
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nConfusion Matrix")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("\nClassification Report")
```

```
print(classification_report(y_test, y_pred))
```

Output :

```
Accuracy: 0.9897142857142858

Confusion Matrix
[[12565  92]
 [  52 1291]]

Classification Report
precision    recall  f1-score   support

   False      1.00     0.99     0.99     12657
    True      0.93     0.96     0.95      1343

 accuracy      0.96     0.98     0.99     14000
  macro avg      0.96     0.98     0.97     14000
 weighted avg      0.99     0.99     0.99     14000
```

Q.3 Modify the program to detect Digit 9 vs Not Digit 9.

Code :

```
# Load Dataset
```

```
mnist = fetch_openml('mnist_784', version=1, as_frame=False)
```

```
X = mnist.data
```

```
y = mnist.target.astype(int)
```

```
# Digit 9 vs Not 9
```

```
y_9 = (y == 9)
```

```
# Split Dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y_9, test_size=0.2, random_state=42
)
```

```
# Train Model
```

```
model = SGDClassifier(random_state=42)
```

```
model.fit(X_train, y_train)
```

```
# Prediction
```

```
y_pred = model.predict(X_test)
```

```
# Results
```

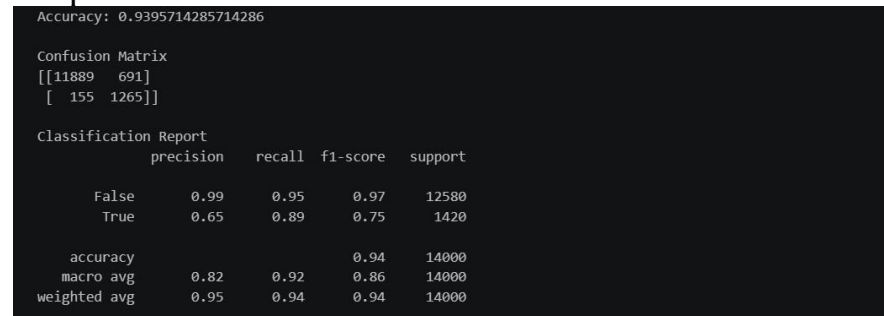
```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Sagar kandari
T010

```
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))
```

```
print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

Output :



```
Accuracy: 0.9395714285714286

Confusion Matrix
[[11889  691]
 [  155 1265]]

Classification Report
              precision    recall  f1-score   support

     False       0.99      0.95      0.97     12580
        True       0.65      0.89      0.75      1420

   accuracy          0.94      0.94      0.94     14000
  macro avg          0.82      0.92      0.86     14000
 weighted avg          0.95      0.94      0.94     14000
```

Q.4 Modify the program to display the first 20 predictions instead of only one prediction.

Code :

```
# Load Dataset
mnist = fetch_openml('mnist_784', version=1, as_frame=False)
```

```
X = mnist.data
y = mnist.target.astype(int)
```

```
# Example: Digit 5 vs Not 5
y_5 = (y == 5)
```

```
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y_5, test_size=0.2, random_state=42
)
```

```
# Train Model
model = SGDClassifier(random_state=42)
model.fit(X_train, y_train)
```

```
# Predict
predictions = model.predict(X_test)
```

```
# Display First 20 Predictions
print("First 20 Predictions:\n")
```

```
for i in range(20):
```

Sagar kandari
T010

```
actual = "Digit 5" if y_test[i] else "Not Digit 5"  
predicted = "Digit 5" if predictions[i] else "Not Digit 5"
```

```
print(f"Image {i+1}")  
print("Actual   :", actual)  
print("Predicted :", predicted)  
print("-----")
```

Output :

First 20 Predictions:

```
Image 1  
Actual   : Not Digit 5  
Predicted : Not Digit 5  
-----  
Image 2  
Actual   : Not Digit 5  
Predicted : Not Digit 5  
-----  
Image 3  
Actual   : Not Digit 5  
Predicted : Digit 5  
-----  
Image 4  
Actual   : Not Digit 5  
Predicted : Not Digit 5  
-----  
Image 5  
Actual   : Not Digit 5  
Predicted : Not Digit 5  
-----  
Image 6  
Actual   : Not Digit 5  
Predicted : Not Digit 5  
...  
Image 20  
Actual   : Not Digit 5  
Predicted : Not Digit 5  
-----
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings...](#)